KNN - Atlas Skilltech University

Nov 7, 2026 . @Razeen

KNN ek simple algorithm hai jo distance ke basis par classify karta hai.

1. KNN - Introduction (Parichay)

K-Nearest Neighbors (KNN) ek supervised machine learning algorithm hai jo **instance-based** ya **lazy learner** hai.

Intuition (Samajhne ka Tarika):

> "Batao mujhe kaun hai tere dost, main bataunga tu kaun hai." Tell me who your friends are, I'll tell you who you are.

KNN ka basic idea:

- Naya data point aaya
- Training data mein uske **K sabse kareeb (nearest) neighbours** dhundho
- Majority vote se class decide karo (Classification) ya average lo (Regression)

Kyun Lazy Learner?

Property	Eager Learner (e.g. Decision Tree)	Lazy Learner (KNN)
Training	Model banta hai	Kuch nahi hota
Prediction	Fast	Slow
Memory	Model save hota hai	Poora dataset yaad rakhta hai
Adaptability	Fixed model	Naye data ke saath adapt

KNN kab use karein?

- Dataset chota ho
- Decision boundary complex ho
- Interpretability chahiye
- Quick baseline model chahiye

Limitations:

- Bade dataset par slow prediction
- Feature scaling zaruri hai

- High dimensions mein performance girti hai (Curse of Dimensionality)

2. Distance Functions (Doori Ke Formulas)

KNN mein sabse important cheez hai **distance** — do points ke beech kitni doori hai?

2.1 Euclidean Distance (Seedhi Doori)

Sabse common distance formula. Ruler se seedhi line ki doori jaisa.

```
d_{Euclidean}(A, B) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}
```

2D mein: `d = sqrt((x2-x1)^2 + (y2-y1)^2)`

Example: Point A = (1, 2), Point B = (4, 6) d = sqrt((4-1)^2 + (6-2)^2) = sqrt(9+16) = sqrt(25) = **5**

2.2 Manhattan Distance (Grid Path / Taxi Cab)

Grid par chalne ki doori. Jaise taxi road par chalti hai — seedha nahi, grid mein.

```
d_{Manhattan}(A, B) = \sum_{i=1}^{n} |A_i - B_i|
```

2D mein: `d = |x2-x1| + |y2-y1|`

Example: Point A = (1, 2), Point B = (4, 6) d = |4-1| + |6-2| = 3 + 4 = **7**

2.3 Minkowski Distance (Generalized Formula)

Euclidean aur Manhattan dono ka generalized version.

```
d_{Minkowski}(A, B) = \left(\sum_{i=1}^{n} |A_i - B_i|^p\right)^{1/p}
```

- **p = 1** => Manhattan Distance
- **p = 2** => Euclidean Distance
- **p = 3** => Minkowski (p=3)

Distance Metrics Comparison

Distance	Formula	p value	Use Case
Euclidean	sqrt(sum of squared diff)	p=2	Continuous data, most common
Manhattan	sum of absolute diff	p=1	Outlier-robust, grid problems

Distance	Formula	p value	Use Case
Minkowski	generalized	p=any	General purpose
Hamming	count of different bits	-	Categorical / binary data

sklearn mein:

```
KNeighborsClassifier(metric='minkowski', p=2)  # Euclidean
KNeighborsClassifier(metric='manhattan')       # Manhattan
KNeighborsClassifier(metric='euclidean')       # Direct Euclidean
```

3. Mathematical Approach (Algorithmic Steps)

KNN kaise kaam karta hai — step by step:

1. **Training Phase:** Poora training data memory mein store kar lo (kuch compute nahi hota)
2. **Prediction Phase:** Naya query point aaya
3. **Calculate Distances:** Query point aur har training point ke beech distance nikaalo
4. **Sort Distances:** Sabse chhoti (nearest) distances pehle rakho
5. **Select K Neighbours:** Top K nearest neighbours chuno
6. **Vote:** Classification — majority class; Regression — average value

From Scratch Implementation

```
import numpy as np
from collections import Counter

def knn_predict_scratch(X_train, y_train, query_point, K):
    # Step 1: Har training point se distance nikaalo
    distances = []
    for i, x in enumerate(X_train):
        dist = np.sqrt(np.sum((np.array(x) - np.array(query_point))**2))
        distances.append((dist, y_train[i]))

    # Step 2: Distance ke hisaab se sort karo
    distances.sort(key=lambda x: x[0])

    # Step 3: K nearest neighbours lo
    k_nearest = distances[:K]

    # Step 4: Majority vote
```

```
    labels = [label for _, label in k_nearest]
    prediction = Counter(labels).most_common(1)[0][0]
    return prediction

# Example use:
# X_train = [[1,2], [3,4], [5,6]]
# y_train = [0, 1, 1]
# query = [2, 3]
# knn_predict_scratch(X_train, y_train, query, K=3)
```

Mathematical Representation

KNN Classification prediction formula:

```
\hat{y} = \arg\max_{c} \sum_{x_i \in N_K(x)} \mathbb{1}[y_i = c]
```

Jahan:

- `N_K(x)` = K nearest neighbours of query point x
- `c` = class label
- `1[y_i = c]` = 1 agar neighbour ki class = c, warna 0

KNN Regression prediction formula:

```
\hat{y} = \frac{1}{K} \sum_{x_i \in N_K(x)} y_i
```

Time Complexity

Phase	Time Complexity	Explanation
Training	O(1)	Kuch nahi hota, bas data store
Prediction (1 query)	O(n * d)	n=training samples, d=features
Prediction with KD-Tree	O(log n * d)	Optimized search

Space Complexity: O(n * d) — poora training data memory mein

4. Choosing K (Best K Kaise Choose Karein?)

K sabse important hyperparameter hai KNN mein.

K ka Effect

K ki Value	Model Behavior	Problem
K = 1	Sirf 1 neighbour dekho	Overfitting — noise ke saath bhagna
K = 3-5	Thode neighbours	Achha balance
K = 15-20	Zyada neighbours	Underfitting — boundary smooth ho jaati
K = n (sab)	Sab ko dekho	Har cheez ek class predict karega

Elbow Method

Step 1: K = 1, 2, 3, ... tak har K ke liye model train karo **Step 2:** Har K ke liye test accuracy nikaalo **Step 3:** Graph plot karo (K vs Accuracy) **Step 4:** Jahan accuracy stable ho jaaye (elbow shape) — wahi best K

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

best_k = 1
best_acc = 0

for k in range(1, 31):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    acc = accuracy_score(y_test, knn.predict(X_test_scaled))
    if acc > best_acc:
        best_acc = acc
        best_k = k

print(f'Best K = {best_k}, Accuracy = {best_acc:.4f}')
```

Rules of Thumb

- K aksar **odd number** choose karo (binary classification mein tie avoid karne ke liye)
- Common starting point: `K = sqrt(n)` jahan n = training samples
- Cross Validation use karo best K ke liye
- Zyada data hoga to thoda bada K theek hai

Overfitting vs Underfitting

$$\text{Small K} \Rightarrow \text{High Variance, Low Bias} \Rightarrow \text{Overfitting}$$

$$\text{Large K} \Rightarrow \text{Low Variance, High Bias} \Rightarrow \text{Underfitting}$$

5. Feature Scaling (KNN ke liye ZARURI!)

KNN distance-based algorithm hai, isliye **feature scaling bahut zaroori hai**. Bina scaling ke, badi values wale features dominant ho jaate hain.

Why Scaling Matters — Example

Maan lo do features hain:

- **Age:** 20 se 60 (range = 40)
- **Salary:** 20,000 se 1,00,000 (range = 80,000)

Bina scaling ke, salary distance dominate karega: `d = sqrt((age_diff)^2 + (salary_diff)^2)` `d = sqrt((5)^2 + (20000)^2)` → salary wins!

Scaling ke baad (0-1 range): `d = sqrt((0.125)^2 + (0.25)^2)` → dono features equal contribute karte hain

Scaling Methods

Method	Formula	Range	Best For
StandardScaler	(x - mean) / std	-inf to +inf (mean=0)	Normal distribution
MinMaxScaler	(x - min) / (max - min)	0 to 1	Bounded features
RobustScaler	(x - median) / IQR	varies	Outliers present

StandardScaler Formula

$$z = \frac{x - \mu}{\sigma}$$

Jahan: mu = mean, sigma = standard deviation

MinMaxScaler Formula

```
x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}
```

Important Rule

- Scaler ko **sirf training data par fit karo**: `scaler.fit_transform(X_train)`
- Test data par **sirf transform karo**: `scaler.transform(X_test)`
- Agar test data par bhi fit karo, to **data leakage** hogi!

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit + Transform
X_test_scaled = scaler.transform(X_test)         # Sirf Transform!
```

6. KNN Classification vs KNN Regression

KNN dono tasks ke liye use ho sakta hai — Classification aur Regression.

Feature	KNN Classification	KNN Regression
Task	Category predict karna	Number predict karna
Prediction	Majority vote	Average of K neighbours
Output	Discrete class label	Continuous value
sklearn Class	KNeighborsClassifier	KNeighborsRegressor
Metrics	Accuracy, F1, Precision	MSE, RMSE, R2 Score
Example	Spam/Not Spam	House Price

Classification Example (Fruit)

```
from sklearn.neighbors import KNeighborsClassifier

knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train_scaled, y_train)
y_pred = knn_clf.predict(X_test_scaled)
# Output: ['Apple', 'Mango', 'Orange', ...]
```

Regression Example (House Price)

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train_scaled, y_train_prices)
y_pred_prices = knn_reg.predict(X_test_scaled)
rmse = np.sqrt(mean_squared_error(y_test_prices, y_pred_prices))
# Output: [45000, 62000, 38000, ...]
```

Regression Formula

```
\hat{y}_{regression} = \frac{1}{K} \sum_{i \in N_K(x)} y_i
```

Classification mein weights='distance' use karne par:

```
\hat{y}_{weighted} = \arg\max_{c} \sum_{i \in N_K(x)} \frac{1}{d(x, x_i)}
\cdot \mathbb{1}[y_i = c]
```

7. Evaluation Metrics (Performance Measure)

Confusion Matrix

Confusion Matrix batata hai model ne kya sahi kiya aur kya galat.

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

- **TP:** Actually positive tha, model ne positive bola. SAHI!
- **TN:** Actually negative tha, model ne negative bola. SAHI!
- **FP:** Actually negative tha, model ne positive bola. GALAT! (Type 1 Error)
- **FN:** Actually positive tha, model ne negative bola. GALAT! (Type 2 Error)

Accuracy

```
\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}
```

Sabse simple metric. Lekin imbalanced data mein misleading ho sakta hai.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

> Jab model ne positive bola, kitni baar sach tha? "Precision mein FP minimize karna chahte hain." Use: Spam detection (False Alarm minimize karo)

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

> Jo actually positive tha, kitne pakad paaye? "Recall mein FN minimize karna chahte hain." Use: Cancer detection (Miss karna zyada costly)

F1 Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision aur Recall ka harmonic mean. Jab dono balance chahiye.

Precision vs Recall Trade-off

Situation	Prefer	Reason
Spam detection	Precision	False alarm annoying hai
Cancer detection	Recall	Miss karna dangerous hai
Equal importance	F1 Score	Balance chahiye

sklearn Implementation

```
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    classification_report,
    precision_score, recall_score, f1_score
)

# Naya tarika (sklearn 1.0+)
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)

# Purana tarika (seaborn)
import seaborn as sns
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True)

# Full report
print(classification_report(y_test, y_pred))
```

8. Curse of Dimensionality (Dimensionality ka Abhishap)

Dimensionality = features ki sankhya (number of features/columns)

Problem Kya Hai?

Jab features bahut zyada ho jaate hain, to KNN ki performance girti hai:

- 2D mein: points close hote hain
- 10D mein: points sparse (door-door) ho jaate hain
- 100D mein: sabhi points ek jaise lagbhag same distance par hote hain

> Jab sabhi points ka distance approximately equal ho, to "nearest neighbour" ka concept meaningless ho jaata hai!

Mathematical Intuition

n-dimensional hypersphere ki volume:

```
V_n(r) = \frac{\pi^{n/2}}{\Gamma(n/2 + 1)} r^n
```

Jab n badhta hai, unit hypercube ka volume mostly corners mein hota hai, center mein nahi.

Practical Impact

Dimensions	Average Distance	Problem
2D	Manageable	KNN achha kaam karta hai
10D	Increases	Performance girti hai
100D+	Near-equal distances	KNN almost random ho jaata hai

Solutions

- **Feature Selection:** Sirf important features rakho
- **Dimensionality Reduction:** PCA (Principal Component Analysis) use karo
- **Feature Engineering:** Domain knowledge se better features banao
- **Alternative Algorithms:** High dimensions mein SVM ya Tree-based models better hote hain

Rule of Thumb

KNN accha kaam karta hai jab:

```
n_{samples} >> 2^{n_{features}}
```

Mtlab: Samples ki sankhya features ki exponential se zyada honi chahiye.

9. Quick Reference Cheat Sheet

KNN Algorithm - Summary

1. Data collect karo
2. Features scale karo (StandardScaler / MinMaxScaler)
3. Train-Test Split karo (stratify=y)
4. Best K dhundo (Elbow Method)
5. KNeighborsClassifier fit karo
6. Predict karo
7. Evaluate karo (Confusion Matrix, Classification Report)

sklearn KNN API

```
# Classification
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=5,          # K ki value
    metric='minkowski',     # Distance formula
    p=2,                    # p=2: Euclidean, p=1: Manhattan
    weights='uniform',      # 'uniform' ya 'distance'
    algorithm='auto',       # 'ball_tree', 'kd_tree', 'brute'
    n_jobs=-1               # Parallel processing
)

# Regression
from sklearn.neighbors import KNeighborsRegressor

knn_reg = KNeighborsRegressor(
    n_neighbors=5,
    metric='euclidean',
    weights='distance'
)
```

Key Formulas Quick View

Formula	Expression
Euclidean Distance	sqrt(sum((Ai - Bi)^2))
Manhattan Distance	sum(
Minkowski Distance	(sum(
StandardScaler	(x - mean) / std
Accuracy	(TP + TN) / Total
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1 Score	2 * (P * R) / (P + R)

Complete Pipeline Template

```
import pandas as pd
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# 1. Data Load
df = pd.read_csv('data.csv')

# 2. Features & Target
X = df.drop('target', axis=1)
y = df['target']

# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Best K (Elbow)
best_k, best_acc = 1, 0
for k in range(1, 31):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    acc = knn.score(X_test, y_test)
    if acc > best_acc: best_acc, best_k = acc, k

# 6. Train Final Model
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)

# 7. Evaluate
y_pred = knn.predict(X_test)
print(classification_report(y_test, y_pred))
ConfusionMatrixDisplay.from_estimator(knn, X_test, y_test)
plt.show()
```

Common Mistakes to Avoid

- Scaling bhool jana (sabse common mistake!)
- Test data par scaler fit karna (data leakage)
- K = 1 rakhna (overfitting)
- Imbalanced data ignore karna
- Cross validation nahi karna

Atlas Skilltech University — Machine Learning Course
KNN Theory Guide — Complete Reference